Cloud access*

Distributed Data Processing Environments

Lab Guide 3

This session aims at creating and disposing of cloud instances. Set up your account, your budget, and your quotas as described in the cloud protocol before starting.

Steps First create an instance from the Cloud Console:

1. Create a new instance selecting:
   - region *us-central1* and zone *us-central1-a*;
   - machine type: e2-micro;
   - operating system type Ubuntu;
   - version 24.04 LTS (x86_64).
2. Before creating it, note the estimated monthly cost.
3. Change the machine type to e2-medium, and then to e2-standard-2, noting the estimate each time. Change the region to *europe-west1* and note it again.
4. Restore the original selection, choose "Create", and wait for the instance to become ready.

Now connect to the instance and work in it:

1. Connect with `gcloud compute ssh`.
2. Install packages and run the program from Lab 1. Copy files with `gcloud compute scp`.
3. Ask the agent what this instance costs per month. Compare its answer with the estimate you noted above, and account for every line item that it omitted.
4. Delete the instance from the console.

Finally, replace all of the above with a description:

1. Create an empty folder and copy into it the `main.tf` from Lab 2.
2. Replace the Docker provider with the Google provider:

```
terraform {
  required_providers {
    google = { source = "hashicorp/google", version = "~> 6.0" }
  }
}

provider "google" {
  project = var.project
  zone    = "us-central1-a"
}
```

*Adversary mode in the second part. Author mode in the last. See AI protocol.

It authenticates with the credentials left by `gcloud auth application-default login`.

3. Replace `docker_container` with a `google_compute_instance`,[1] carrying the zone, machine type, and image you selected by hand above.
4. Add a `scheduling` block so that the instance deletes itself after 8 hours.
5. Predict what `tofu plan` will report, then run it, and then apply.
6. Connect to the instance and confirm that it is the one you described.
7. Change the machine type in the file, and explain whether `plan` updates the instance or replaces it.

Now let the description do the work, on more than one machine:

1. Make the instance set itself up, instead of being set up by hand:

```
metadata_startup_script = templatefile("startup.sh.tftpl", {
  team = "Bayern Munich"
})
```

2. Write `startup.sh.tftpl`: a shell script that installs Python, downloads the dataset, and runs a program counting the goals of one team. Where the script says `${team}`, OpenTofu puts the value in before the script ever reaches the machine. Ask the agent to write it, then say what you would have done differently.
3. Apply, and read the answer without connecting to anything:
`gcloud compute instances get-serial-port-output` *name*
4. Now ask for one machine per team:

```
variable "teams" {
  type    = list(string)
  default = ["Bayern Munich", "Bayer Leverkusen", "Augsburg"]
}

resource "google_compute_instance" "run" {
  for_each     = toset(var.teams)
  name         = "adpd-${replace(lower(each.key), " ", "-")}"
  ...
}
```

5. Add a fourth team. Predict what `plan` will report, and only then run it.
6. Ask the agent to rewrite the same description with `count` instead of `for_each`. Remove a team from the **middle** of the list, and explain what `plan` now proposes and why the two are not the same.
7. Collect the answers of all the machines into one. Say why that was awkward, and what would have made it easy.

At the end of the session, run `tofu destroy` and follow the teardown checklist in the cloud protocol.

Learning Outcomes Create and dispose of cloud instances, by hand and from a description. Relate instance type and location with cost. Provision an instance from its description, without connecting to it. Describe a set of similar instances without repeating the description.

[1] `https://registry.terraform.io/providers/hashicorp/google/latest/docs/resources/compute_instance`